

DEVOPS PROJECT DOCUMENTATION

VProfile on Docker

توثيق كامل للمشروع وطريقة الـ Deploy من الصفر

Multi-stage build · Docker Compose · Tomcat · MySQL · RabbitMQ ·
Memcached

Tomcat 9 · JRE 8

Tier Java App-3

Docker Compose

Multi-stage Dockerfile

ALnagib

YOUSEF · DEVOPS ENGINEER

WHAT IS VPROFILE

1

نظرة عامة على المشروع

VProfile هو تطبيق **Java (WAR)** يشتغل على **Tomcat**، ومبني على معمارية **Tier-3** كلاسيكية: طبقة العرض (المستخدم/المتصفح)، طبقة التطبيق (Tomcat)، وطبقة البيانات والخدمات المساعدة (MySQL, RabbitMQ, Memcached). الفكرة الأساسية إننا حوّلنا نفس المشروع اللي كان شغال **On-prem** على 5 سيرفرات منفصلة (Vagrant VMs) لبيئة **Container-based** باستخدام **Docker Compose** — نفس ال architecture بالظبط بس كل VM بقى container.

STACK COMPONENTS

2

مكوّنات ال Stack

db01 • mysql:8.0

قاعدة البيانات

قاعدة بيانات **MySQL** اسمها **accounts** على البورت **3306**، مع volume دائم و seed تلقائي من ملف **SQL**.

app • tomcat:9-jre8

طبقة التطبيق

سيرفر **Tomcat** يستضيف ال WAR باسم **ROOT.war** على البورت **8080**. مبني من ال source عن طريق multi-stage build.

mc01 • memcached:1.6

طبقة ال Cache

Memcached لتسريع القراءة وتقليل الضغط على قاعدة البيانات، على البورت **11211**.

rmq01 • rabbitmq:3-mgmt

Message Broker

RabbitMQ لل messaging على **5672**، مع واجهة إدارة على **15672**.

SERVICE DISCOVERY

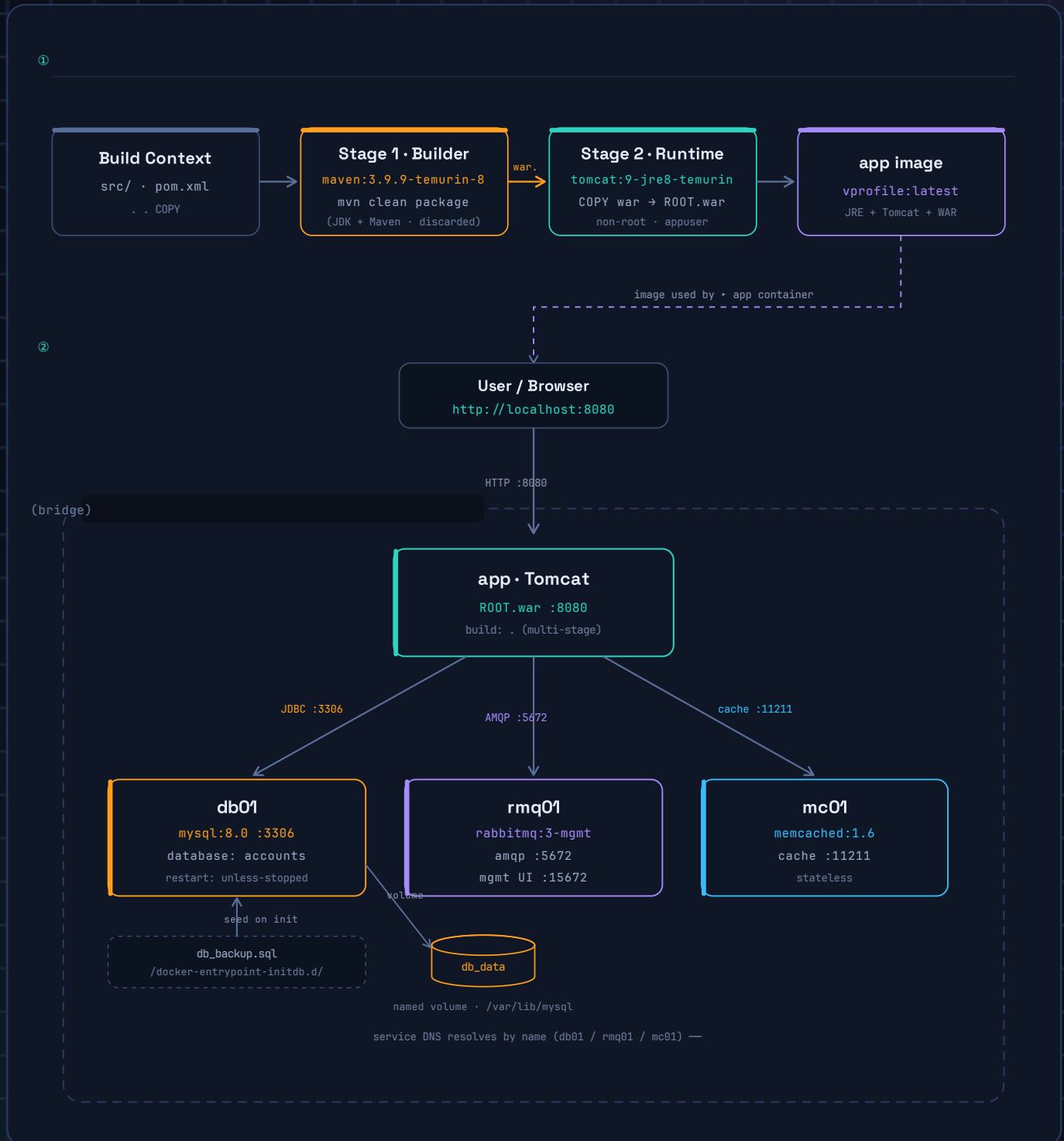
الأربع خدمات بيلاقوا بعض عن طريق أسماء ال **services** (**db01**, **rmq01**, **mc01**) اللي بتتحول تلقائياً ل hostnames جوّه شبكة Docker الداخلية (Internal DNS). عشان كده ملف **application.properties** مكتوب فيه نفس الأسماء بالظبط.

END-TO-END DIAGRAM

الرسمه المعمارية الكاملة

3

الرسمه بتوضح المرحلتين: **مرحلة البناء** (فوق) اللي بتنتج الـ image، و**مرحلة التشغيل** (تحت) اللي بتربط الأربع containers جوفه شبكة واحدة.



HOW THE IMAGE IS BUILT

كيف يُبنى الـ Image

4

الـ **Dockerfile** يستخدم **Multi-stage Build** — مرحلتين: الأولى تبني وترمي نفسها، والثانية هي التي يشتغل فعلياً.

مرحلة الـ **Maven** — Builder

1

يتأخذ الـ base image **maven:3.9.9-eclipse-temurin-8**، بتعمل **COPY . .** الـ source، ويشغل **mvn clean package** عشان تطلع ملف **target/vprofile-v2.war**. المرحلة دي فيها JDK كامل + Maven، حجمها كبير، لكنها بتترمي بعد ما تخلص شغلها.

مرحلة الـ **Tomcat** — Runtime

2

بتبدأ من **tomcat:9-jre8-temurin** (JRE فقط، من غير compiler = أصغر وأمن). بتنشئ مستخدم **appuser** غير **root**، بتمسح تطبيقات Tomcat الافتراضية، وتنسخ الـ WAR من المرحلة الأولى باسم **ROOT.war** عشان يشتغل على المسار **/** مباشرة.

```
# ----- Build stage ----- FROM maven:3.9.9-eclipse-temurin-8 AS builder WORKDIR
/build COPY . . RUN mvn -B clean package # → vprofile-v2.war # ----- Runtime stage ----
----- FROM tomcat:9-jre8-temurin RUN groupadd -r appgroup && useradd -r -g appgroup appuser
RUN rm -rf /usr/local/tomcat/webapps/* COPY --from=builder --chown=appuser:appgroup \
/build/target/vprofile-v2.war /usr/local/tomcat/webapps/ROOT.war USER appuser EXPOSE 8080
CMD ["catalina.sh", "run"]
```

?WHY MULTI-STAGE

الـ image النهائي بيحتوي على **Tomcat + JRE + الـ WAR فقط** — من غير Maven ولا JDK ولا الـ source code. النتيجة: حجم أصغر، سطح هجوم (attack surface) أقل، وأمان أعلى لأن الـ container بيشتغل بمستخدم غير **root**.

STEP-BY-STEP DEPLOYMENT

ال Deploy من الصفر

5

المتطلبات — Prerequisites

1

تأكد إن عندك **Docker Engine** و **Docker Compose v2** متنصّين وشغّالين.

```
$ docker --version $ docker compose version
```

ادخل على مجلد المشروع

2

كل الأوامر بتنشغل من جوه فولدر المشروع الي فيه ال **Dockerfile** و **docker-compose.yml**.

```
$ cd sourcecodeseniorwr
```

ابن وشغل ال Stack كامل

3

الأمر ده بيبنّي ال app image من ال Dockerfile، بيسحب باقي ال images الجاهزة، ويشغل الأربعة مع بعض.

```
$ docker compose up --build -d
```

اتأكد إن كل حاجة شغّالة

4

لازم تشوف الأربع containers في حالة **Up**.

```
$ docker compose ps $ docker compose logs -f app
```

افتح التطبيق

5

من المتصفح ادخل على العنوان ده — هتلاقي صفحة تسجيل الدخول.

```
http://localhost:8080 # app (login) http://localhost:15672 # RabbitMQ mgmt (guest/guest)
```

DEPLOY SEQUENCE

بسبب **depends_on** بيقوم **db01**, **rmq01**, **mc01** الأول، وبعدين **app**. أول ما **db01** يقوم، بياخد ملف **db_backup.sql** المعمول له mount في **/docker-entrypoint-initdb.d/** ويعمله import تلقائي لجدول قاعدة البيانات.

إيقاف وتنظيف

6

بتوقف وتنظف ال containers، وإضافة **-v** بتسمح ال volume (بيانات قاعدة البيانات) كمان.

```
$ docker compose down # stop + remove containers $ docker compose down -v # + remove db volume
```

WHAT'S NEEDED VS EXTRA

6

بنية المشروع – اللازم والزيادة

مش كل الملفات الموجودة في الفولدر ضرورية وقت الـ build/run. دي الخريطة:

sourcecodeseniorwr/ | Dockerfile # ✓ يبنى الـ app image | docker-compose.yml # ✓ يشغل
 تعريف # ✓ pom.xml | build | يمنع الملفات الوسخة من الـ # ✓ .dockerignore | services الأربع
 الـ Maven build | src/ | | main/resources/ | | application.properties # ✓ ربط الـ app
 build مخرجات • 105MB # 'target/ | بيانات قاعدة البيانات # ✓ db_backup.sql | بالخدمات
 (اختياري) # .git/ | فولدر فاضي بالغلط - احذفه × # db/db_backup.sql/ | قديم - امسحها
 فقط version control

extra / removable

ملفات زيادة

target/ (بيتبني من جديد جوه الـ Dockerfile) والفولدر
 الفاضي db/ .
 .dockerignore

required

ملفات لازمة

الـ Dockerfile, الـ compose, الـ .dockerignore, الـ
 pom.xml, مجلد src/, وملقي
 application.properties و db_backup.sql

CLEANUP

لتنظيف المشروع وتصغير حجمه: mvn clean أو rm -rf target/ لمسح مخرجات البناء، و rm -rf db/ لمسح الفولدر
 الفاضي. الـ .dockerignore أصلاً بيستثني target/, .git, .env, *.key من الـ build context.

MIGRATION MAPPING

7 من ال On-prem إلى Docker

7

نفس المعمارية، بس استبدلنا كل VM ب container. الأسماء انتقلت زي ما هي عشان الكود ما يتغيرش.

DOCKER COMPOSE	ON-PREM (VAGRANT VMS)	المكوّن
db01 – mysql:8.0	db01 – VM • MariaDB	قاعدة البيانات
mc01 – memcached:1.6	mc01 – VM • Memcached	Cache ال
rmq01 – rabbitmq:3	rmq01 – VM • RabbitMQ	Message Broker
app – multi-stage image	app01 – VM • tomcat.sh	التطبيق
– (غير موجود حالياً)	web01 – VM • Nginx	Reverse Proxy
Docker internal DNS	hostmanager	اكتشاف الأسماء
ثواني • موارد أقل	دقائق + RAM كبير	وقت التشغيل

GOTCHAS & NOTES

8 ملاحظات ونقاط مهمة

DEPENDS_ON ≠ READY

ال `depends_on` بيستأى ال container **يقوم** بس، مش يبقى **جاهز**. يعني ممكن ال app يحاول يتكلم مع MySQL قبل ما يخلص initialization، فتظهر أخطاء اتصال في أول 10–20 ثانية ثم تختفي. الحل الأنظف: إضافة `healthcheck` ل db01 مع `condition: service_healthy`.

MISSING PIECES VS ON-PREM

مفيش `Nginx reverse proxy` (بتوصل Tomcat على 8080 مباشرة)، ومفيش `Elasticsearch` رغم إن `application.properties` لسه فيه إعدادات `vprosearch01` — فخاصية البحث مش هتشتغل، لكن التطبيق نفسه بيقوم عادي.

NEXT STEPS • PRODUCTION HARDENING

نقل ال passwords من ال compose لـ `.env` / Docker secrets • إضافة `healthcheck` لكل خدمة • إضافة Nginx container كـ reverse proxy • تثبيت إصدارات ال images (tags ثابتة) • إضافة `restart` policies • فصل الشبكات (frontend/backend networks).